

ML (&DL) 理论基础

我仍然以吴恩达的视频课为主线，中间穿插了我的思考和数学知识补漏，大部分文字和公式为笔者手打，有些偏专业的叙述采摘至周志华的西瓜书，还有一些来自AI工具的补充

什么是机器学习

1. 进行一个deep seek的类比：

- i. 传统编程方法：我们会尝试编写一套精确的规则，例如：“它有一个圆脸、两只尖耳朵、胡须、四条腿、一条长尾巴...” 但我们会发现，有很多例外（有的猫没尾巴，有的狗也符合这些特征），规则会变得极其复杂且不准确
 - ii. 机器学习方法：我们给孩子看成千上万张不同猫的图片（有各种品种、颜色、姿态），并告诉他“这些都是猫”，然后我们再给他看一些狗、汽车、杯子的图片，并说“这些都不是猫”，通过反复观察和练习，孩子的大脑会自己归纳出“猫”的特征，最终即使看到一只从未见过的猫，他也能认出来
 - 机器学习模型就像这个孩子，数据（图片）就是它的教材，学习过程就是它大脑归纳规律的过程
- 核心：没有明确编程，自主学习

2. 应用：

- i. 推荐系统：Netflix、淘宝、抖音的个性化推荐
- ii. 自然语言处理：智能助手（Siri, Alexa）、机器翻译、聊天机器人
- iii. 计算机视觉：人脸识别解锁、医疗影像分析（诊断疾病）、自动驾驶汽车
- iv. 金融风控：检测信用卡欺诈交易
- v. 预测性维护：预测工业设备何时需要维修

3. 辨析：

需要明确的是，机器学习的自主学习体现在不断优化参数，而不是自主选择模型，就像你告诉机器，我们用线性函数来拟合函数，它就不会突发奇想，选择用正弦函数来拟合函数

4. 更深一点（这是我事后对于第三点的补充）：

那就是在后期的理论学习中我们接触到的神经网络：这更进一步。神经网络的结构（有多少层、每层有多少神经元）是人事先设计的。但网络中的所有权重和偏置参数都是机器从数据中学习得来的。神经网络强大的原因在于，它可以通过这些简单的神经元组合，自动逼近极其复杂的非线性函数形式，而不需要人事先精确地规定这个函数长什么样。但归根结底，它的基础结构（层级、激活函数等）仍然是人为设定的

监督学习和无监督学习（浅谈）

1. 机器学习分为两类：surpervise & unsurpervise
2. (Surp)**监督学习**：我自己的话来说，就是：映射关系 $X \rightarrow Y$ 十分明确，人为输入大量X，并明确每一个X的对应的输出Y，模型学习了这些映射关系，总结出一种可操作的函数对应模型，在我们新输入一个从未见过的X，它根据自己建立的模型尝试输出一个令我们满意的、合适的Y予以回应
3. (Unsurp) **无监督学习**：我们并没有给出明确的Y，相当于老师们不给学生演示例题了...
无监督学习分为多类：
 - i. 聚类：原理大概是让模型自己去玩找不同和找相同，然后按照他自己决定的标准把对象分为几个组或簇。他的应用也比较多，比如新闻把相关的实列放在一起（通过检索一样的关键词：9.3日，阅兵...）；生物学中对比基因（DNA微阵列）的相同之处将不同的个体进行性状归类；视屏推送的视屏类型和标签；
 - ii. 异常检测：检测异常事件：比如金融诈骗检测；
 - iii. 降维压缩大数据集，并尽可能地减少损失数据的损失

线性回归模型

我的预理解，大概类似于我们高中的一元线性拟合

线性回归模型，是属于回归类型的一种，相对于回归模型，我们还有分类模型（存在离散的有限可能输出，最常见的比如1&0）

- 单线性拟合：

$$f_{w,b}(x) = wx + b = y - \hat{}$$

上面公式中, x 称为**输入变量**, y 称为**特征**或者输出特征, f 即为函数

我们用 $(x^{(i)}, y^{(i)})$ 表示第i组训练数据, 这样的数据集合一起称为**训练数据集**, 训练数据个数用m表示; 求出的Y-han是我们的**预测值**; W我们称之为**权重**; b我们称之为**截距**

成本函数（cost function）

用于衡量我们模型的拟合程度的一个新的函数（以平方误差成本函数为例）：

$$J_{w,b} = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

类似于高中阶段的线性拟合 r^2

在实际操作中，我们希望最小化J函数，记作

$$\text{minimize}_{w,b} J(w,b)$$

线性拟合的可视化：类似于求偏导

梯度下降

1. 用于最小化J函数的一种算法操作

- 前提是每一次从初始(w,b)的值开始，不断改变w和b的值，从而减小J函数的值。根据可视化的处理，以双元函数为例，我们把可视化的“小丘陵”看作是地图表面，我们的数据就像一个探险家，要去寻找这片地区的最低点。我们选取的初始取值可能就是这位探险家的初始营地。他要通过一系列的操作到达做地点，他每一次操作就是一次取值，也是一次下降。而我们通过最短路径下降至最低点的算法就是梯度下降。
- 当然以上是我抛开高数来说的浅显的理解。其实一次梯度下降的到局部最小值可能并不是我们要找到的整体最小值。我也很容易想到这是为什么，因为函数的极值不一定是函数的最值。那么很显而易见的，我认为梯度下降就是一种高效的求多元函数极小值的机械方法。但具体怎么操作和数学定义我其实不太清楚，之后再看看吧
- 之后来看：

思考： 在ML最经典的梯度下降求成本函数最小值的时候，所得值可能是局部最小值也就是某个极值，我们怎么判断是整体最小值还是局部最小值？怎么样优化算法呢？

回答： 不是关注解的大小，而是去关注解的质量

- 许多局部最小值的“质量”是相似的：对于大规模的过参数化模型（如深度神经网络），许多局部最小值在测试集上的性能表现非常接近。也就是说，损失函数值虽然不同，但泛化能力可能差别不大。
- “平坦的”局部最小值通常泛化更好：与尖锐的局部最小值相比，位于平坦区域的局部最小值对参数扰动不敏感，通常被认为具有更好的泛化能力。
- 我们更担心的是鞍点：在高维空间中，鞍点（梯度为零但既不是最小值也不是最大值的点）的数量远远多于局部最小值。梯度下降可能会在鞍点附近停滞很长时间。

如何评估解的质量：

- 多次随机初始化（多次重启）：
这是最常用且最重要的方法。从不同的随机初始点开始运行优化算法多次。如果大多数运行都收

收敛到相似的成本函数值，那么这个值很可能是一个“有代表性”的好的局部最小值（即使不是全局最小）。如果结果差异很大，说明损失函数地形复杂，需要更仔细的调优。

- 验证集性能：

我们最终关心的不是训练损失有多低，而是模型在未见数据上的表现。记录每次训练后模型在独立验证集上的误差。通常，训练损失较低且验证集性能良好的解，就是一个“好”的解。验证集性能是最终的评判标准。

- 观察训练动态：

损失曲线：观察训练损失下降的轨迹。一个“好”的优化过程通常会有平滑、稳定的下降，最终收敛到一个平台。

梯度范数：监控梯度向量的范数。在最小值（局部或全局）附近，梯度范数应该趋近于零。如果它提前稳定在一个较大的值，可能陷入了鞍点或高原。

如何优化算法以找到更好的解：

1. 优化器（算法）层面的改进

1. 带动量的梯度下降：

a. 动量法：引入速度变量，使更新方向不仅取决于当前梯度，还积累了过去梯度的指数加权平均。这有助于在稳定方向（朝向最小值）上加速；在梯度方向振荡的维度上减少震荡。最重要的：拥有足够的“惯性”冲过一些较浅的局部最小值和鞍点。

b. Adam：结合了动量（一阶矩估计）和自适应学习率（二阶矩估计）的算法，是目前最流行的默认优化器，对许多问题都能稳健地找到不错的解。

c. 自适应学习率方法：AdaGrad, RMSProp, Adam 等。它们为每个参数调整学习率，在稀疏梯度上加大更新，在频繁出现梯度上减小更新。这有助于在崎岖的损失地形中更智能地导航。

2. 学习率策略

a. 学习率衰减：随着训练进行，逐渐减小学习率。早期大步探索，后期小步微调，有助于稳定地收敛到最小值点（更平坦的区域）。

b. 热身：训练初期使用较小的学习率，逐步增大，有助于在训练初期稳定模型。

c. 周期性学习率：如 Cyclical LR 或 SGDR，让学习率在一个区间内周期性地变化。这被认为是一种非常有效的“逃离局部最小值”的技巧。当学习率变大时，优化器有可能跳出当前的局部极小盆地，落入另一个可能更好的盆地。

3. 模型与训练技巧

a. 批标准化：它不仅加速训练，更重要的是极大地改善了损失函数的地形，使地形更加平滑、更容易优化，减少了陷入糟糕局部最小值的可能性。

b. 残差连接：如在 ResNet 中，使得梯度流动更顺畅，让极深的网络也能被有效训练，间接改善了优化问题。

c. 随机正则化：Dropout 在训练中随机丢弃神经元，相当于在每次更新时优化一个略微不同的子模型。这可以防止优化器过于“拟合”到某个特定路径的尖锐最小值中，鼓励寻找更鲁棒、更平坦的最小值。当然随机深度、数据增强等也有类似效果。

4. 高级策略

a. 模型集成：训练多个独立初始化的模型，然后平均它们的预测。这直接规避了“把所有鸡蛋放在

一个篮子里”的风险，即使单个模型落在次优解，集成后的结果也往往更稳定、更优。

b. 快照集成：利用周期性学习率，在一个训练周期内保存多个“快照”模型，然后将它们集成。成本远低于训练多个独立模型。

c. 课程学习：先从简单的样本或任务开始训练，再逐步增加难度。这有助于引导优化器走向一个更优的区域。

2. 从数学上来理解

$$w = w - a \frac{d}{dw} J(w, b)$$

其中等号是赋值符号，a(其实是 α)，我们称之为学习率（范围通常在 (0, 1)），这个可以理解为你下山时的步子的大小，如果 α 过于大，那么你下山就会非常快，最后面是J函数的导数项，可以理解为你取步子的前进方向

同理，取新的b:

$$b = b - a \frac{d}{db} J(w, b)$$

重复以上步骤，直到算法收敛，即达到局部最小值的时候，w和b不在跟随我们的取值在显著变化（其实还是好理解，就是导数项都取0的时候函数取到极值）

- 值得提醒的是，和高中物理公式取导数一样，我们需要同时更新w和b元，即赋值号右边的式子都表示的是未作更改之前的w和b的表达式

3. 关于偏导数浅谈

本人对于偏导数的了解很少，这里做一个浅显的学习记录

从最基础的开始，就当作是提醒吧

i. $f'(x) = \frac{d}{dx} f(x) = Df(x) = D_x f(x) = \frac{dy}{dx}$

ii. 复合函数求导法则略去

iii. 偏导数 $y = f(x_1, x_2, x_3, \dots, x_n)$

求y关于第 x_i 的偏导数为：

$$\frac{\partial y}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, x_2, x_3, \dots, x_i + h, \dots, x_n) - f(x_1, x_2, x_3, \dots, x_n)}{h}$$

iv. 关于梯度：

$f(x)$ 关于 x 的梯度是一个包含了n个偏导数的向量，其中 x 也是一个n为向量

v. 复合函数求偏导的链式法则

对于可微函数 $y = f(u)$ 和 $u = g(x)$, 类推可以有，设y有m个变元 u_i , 同时每一个 u_i 有变量 $x_1, x_2, x_3, \dots, x_n$ 对于任意的 $i = 1, 2, 3, 4, \dots, n$ 都有法则：

$$\frac{\partial y}{\partial x_i} = \frac{\partial y}{\partial u_1} \frac{\partial u_1}{\partial x_i} + \frac{\partial y}{\partial u_2} \frac{\partial u_2}{\partial x_i} + \dots + \frac{\partial y}{\partial u_m} \frac{\partial u_m}{\partial x_i}$$

4. 关于学习率 α

须注意的是， α 过大过小都会使模型学习效率下降：过小的话操作次数太多；过大的话反而可能反其道而行之，离最小值越来越远，即梯度下降是发散的

思考：那该如何确定我们需要的 α 值呢？我们都知道我们拟合的函数一般是来连续的（至少在线性回归中是这样的），而明显，在这种梯度算法的每一步都是在确定的离散的点之间的跳跃，如果很不巧地我们一直在极值附近震荡，该怎么结束我们的算法呢？——我的第一反应是像二分法一样给定一个精确度，由此来确定结束区间。但是如果我们连最值都算不出来，精确度的基本比较点怎么取出呢？——我的第二反应是，与前一次得到的J函数进行比较大小，那么这样即使我们可以得到一个近似局部最小值，可是这样我们又怎么说明精确度呢？难道要取大量 α 吗，显然这样浪费了算力

deep seek的解释：

i. 如何选取学习率 α ：

1. 经验值与网格搜索 (Empirical > Values & Grid Search):

- 初始尝试：对于许多问题，学习率设置在 0.01, 0.03, 0.1, 0.3, 1 等数量级是一个不错的起点
- 系统化选择：更科学的方法是进行学习率扫描（Learning Rate Sweep）或网格搜索（Grid Search）。可以在一个较大的范围内（如 [0.0001, 1]）选择一系列呈**对数**尺度分布的学习率，分别运行少量次数的迭代（比如50-100次），然后绘制成本函数 $J(\theta)$ 随迭代次数的变化曲线
- 最优区域：那条曲线下降最快且最终稳定在一个较低值的学习率，通常就是最佳选择
- 排除法：曲线下降缓慢说明 α 太小；曲线剧烈震荡甚至上升说明 α 太大。

2. 自适应学习率算法 (Adaptive Learning Rate Algorithms)

这是现代机器学习中的 **标准做法**。这些算法不再使用全局固定的 α ，而是为每个参数 θ_i 动态地调整学习率。它们能自动适应损失函数的landscape，在平坦处增大步长，在陡峭处减小步长，从而大大加快收敛速度并减少震荡。绝对应该了解这些算法：

- **Adagrad**：为频繁更新的参数减小学习率，为不频繁更新的参数增大学习率。
- **RMSprop**：Adagrad的改进版，解决了其学习率过早过快衰减的问题。
- **Adam** (Adaptive Moment Estimation)：这通常是默认的首选算法。它结合了RMSprop和动量的思想，在实践中表现非常出色，对初始学习率的选择相对不敏感（通常默认的 0.001 或 0.0001 就能工作得很好）。
- 结论：对于简单的线性回归，您可以从 0.01 开始尝试。但对于任何严肃的项目，直接使用 Adam等自适应优化器是最高效、最可靠的选择，它几乎完全避免了手动 tuning α 的痛苦

ii. 如何结束算法？—— 收敛判定 (Convergence Criteria)

a. 根据成本函数 $J(\theta)$ 的变化判定 (与您的第二反应相似)

这是最常用、最直接的方法。我们并不需要知道真实的最小值是多少，我们只关心我们的优化过程是否已经稳定，即参数更新带来的收益已经微乎其微。

- 方法：在每次迭代中，计算当前代价函数的值 $J(\theta^k)$ 和前一次的值 $J(\theta^{k-1})$ 的绝对差或相对差。
- 停止条件：设定一个非常小的阈值 ϵ (epsilon, 例如 $1e-5$, $1e-6$, $1e-7$)。
- 绝对差： $|J(\theta^k) - J(\theta^{k-1})| < \epsilon$
- 相对差： $|J(\theta^k) - J(\theta^{k-1})| / |J(\theta^{k-1})| < \epsilon$ (更常用，因为它考虑了J本身的大小)

- 如何说明精确度？：这里的精确度 ϵ 指的是不是与真实最小值的距离，而是优化的精确度。它意味着“我认为连续两次迭代带来的改进已经小到可以忽略不计，因此宣布收敛”。 ϵ 越小，您得到的解就越接近当前优化路径所能达到的“最优点”，但所需的迭代次数也越多。这是一个计算精度和计算成本之间的权衡

b. **根据参数 θ 的变化判定**（与您的第一反应类似）

同理，我们也可以监测参数本身的变化。

方法：计算当前参数向量 θ^k 和前一个参数向量 θ^{k-1} 之间的欧几里得范数（即距离）。

- 停止条件： $\|\theta^k - \theta^{k-1}\| < \epsilon$

含义：如果参数每次更新的大小已经小于阈值，说明参数已经基本稳定，可以停止。

c. **设置最大迭代次数** (Max Iterations)

这是一个必须设置的安全措施，以防止算法因不收敛（如学习率过大导致一直震荡）或其他问题而无限循环。

如此看来，我们的自主思考并没有太大问题，但其中的数学操作流程肯定更加复杂。这里暂时先不做更多的思考了

5. 欧几里得范数

i. 定义

对于一个在 n 维实数空间 $\mathbb{R}^n$ 中的向量 x ，其坐标为 $(x_1, x_2, \dots, x_n)$ ，它的欧几里得范数定义为：

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

它也是勾股定理在 n 维空间上的推广，特别地：

- 在二维空间 ($\mathbb{R}^2$)：向量 $v = (x, y)$ 的欧几里得范数就是

$$\|v\| = \sqrt{x^2 + y^2}$$

，这正是计算平面上一点到原点距离的公式

- 在三维空间 ($\mathbb{R}^3$)：向量 $v = (x, y, z)$ 的欧几里得范数是

$$\|v\| = \sqrt{x^2 + y^2 + z^2}$$

即计算三维空间中一点到原点的距离

ii. 与其他范数的关系

欧几里得范数是更一般的 L^p 范数的一个特例（当 $p=2$ 时）

- L^1 范数（曼哈顿范数）： $\|x\|_1 = |x_1| + |x_2| + \dots + |x_n|$ 。想象在城市网格中行走，不能走对角线，只能沿街道走
- L^2 范数（欧几里得范数）： $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$ 。就是我们这里讨论的，两点之间的直线距离
- L^∞ 范数（切比雪夫范数/最大值范数）： $\|x\|_\infty = \max(|x_1|, |x_2|, \dots, |x_n|)$ 。取向量所有分量中绝对值最大的那个

iii. 应用

欧几里得范数是科学和工程中最基础、最常用的工具之一，应用极其广泛：

- **机器学习:**

k-最近邻 (k-NN) 算法 中，常用欧几里得距离来衡量数据样本之间的相似度

k-均值聚类 算法 中，通过计算数据点到簇中心的欧几里得距离来进行聚类。

损失（即我们的成本函数）函数，如均方误差，本质上就是基于欧几里得距离的平方

- 计算机图形学: 计算物体的大小、光源的距离、碰撞检测等
- 信号处理: 将一个信号视为高维向量，用其范数来表示信号的强度或能量
- 物理学: 计算力、速度、位移等物理矢量的大小
- 优化问题: 在最小二乘法等问题中，核心就是最小化欧几里得范数（距离）

6. 批量梯度下降

计算机使用的常用方法

多特征

1. 公式:

$$f_{(w,b)}(x) = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$

$$\text{行向量: } \vec{w} = [w_1, w_2, w_3, \dots, w_n]$$

$$\text{行向量: } \vec{x} = [x_1, x_2, x_3, \dots, x_n]$$

$$f_{(w,b)}(\vec{x}) = \vec{x} \cdot \vec{w} + b$$

以上的模型我们称之为多元线性回归

2. 关于向量化

局部代码


```
w = np.array([1.0, 2.5, -3.3])
b = 4
x = np.array([10, 20, 30])
f = np.dot(w, x) + b
# np.是使用了numpy dot function进行了两个向量之间的点积操作而实现向量化；其实这段代码也可以用循环结构，顺
```

向量化的优势体现在：

- 直接在计算机的硬件上同时获取我们保存的向量w和x的所有的值，并同时操作相对应的元，一步就完成了点积操作，提高了编译效率

3. 实现多元线性回归的向量化梯度下降

补充：关于正规方程（normal equation）：

一种仅适用于线性回归的算法，特点是无法推广，无需迭代

这种向量化梯度下降的方法即把公式进行一个推广：

$$w_n = w_n - a \frac{d}{dw_n} J(w_n, b)$$

特征缩放

1. 必要性：

如果我们的不同特征的取值的范围存在较大的差异，那么其对应的权重的分布也会存在较大差异，比如对于二元变量来说，在 w_1 和 w_2 组成的二维等高线来看，椭圆就会非常瘪，那么很有可能梯度下降过程中，在到达最小值的路径之前来回跳跃多次，导致学习效率下降

2. 操作：

对于多个特征有一定手段放缩到基本一致的取值区间，那么特征权重在进行梯度下降的时候就会更加清晰明了（少走弯路），进而提高学习效率

3. 具体操作

- 均值归一化：简单来说其实类似于高中一元线性拟合中的中心化（标准化），即使特征值均匀分布在每个象限内，比如对于第一个特征数据集：

$$x_1 = \frac{x_1 - \bar{x}_1}{\max x_1 - \min x_1}$$

- Z-分数归一化：

$$x_1 = \frac{x_1 - \bar{x}_1}{\sigma_1}$$

其中 σ_1 为特征数据集的标准差

再谈梯度下降收敛判定和学习率

这个问题已经在学习率板块中我的思考已经提到过

1. **学习曲线判定**
2. **自动收敛测试**（上文已经提及，不在赘述）
3. **具体分析：**

图像如果不是严格的单调下降，说明我们的公式存在问题或者 α 取得过大，可以极小化 α 然后不断尝试调试 α 的值来提高学习速度

多项式回归

1. **公式体现：**

$$f(x) = w_1x^n + w_2x^{n-1} + \dots + w_nx$$

2. 我的思考：多项式回归是线性回归吗，多项式回归是单线性回归吗？即特征不止一个时的多项式回归存在不存在，若存在，它属于线性回归吗？

解答：

- **多项式回归**（以二次为例）：

$$y = \beta_0 + \beta_1x + \beta_2x^2$$

虽然我们有了 x^2 项，但方程关于参数 $\beta_0, \beta_1, \beta_2$ 仍然是线性的。我们可以把 x 和 x^2 看作是两个不同的特征

- 当我们有多个特征时（例如 x_1, x_2 ），多项式回归可以包含：

高次项：例如 x_1^2, x_2^3

交互项：不同特征相乘的项，例如 $x_1 x_2$ ，这可以捕捉特征之间的协同效应

例如，一个包含两个特征 (x_1, x_2) 的二次多项式回归模型可能看起来像这样：

$$y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \beta_3x_1^2 + \beta_4x_2^2 + \beta_5x_1x_2$$

在这个方程中：

β_0 是截距

β_1, β_2 是线性项的系数

β_3, β_4 是平方项的系数

β_5 是交互项的系数

Scikit-learn 中的 PolynomialFeatures 工具就是专门用来为多元线性回归自动生成这种多项式特征和交互特征的

分类

1. 二分类:

输出只有两个可能的类别和两个可能的分类,常见的一般是 (1, 0) 分类, 即负类和正类

2. 逻辑回归:

i. 运用最广泛的分类算法之一

ii. 图像显示: 可以想象一下在高中生物种群密度的“S”形的增长曲线, 在这里我们用这条曲线, 将可视化的二维坐标轴分成了两部分 (后面我再学习严谨的数学表达式)

iii. 核心创新在于: 它没有直接去预测类别, 而是去预测一个样本属于某个类别的概率

iv. 数学表达:

a. 必要性: 为了将线性回归的输出 (一个任意实数) 压缩到一个表示概率的区间 $[0, 1]$ 中, 逻辑回归引入了一个关键的函数——**Sigmoid函数**

b. Sigmoid 函数的公式为:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

它的图像是一条优美的S形曲线:

当 z 趋近于正无穷时, $\sigma(z)$ 无限接近于 1;

当 z 趋近于负无穷时, $\sigma(z)$ 无限接近于 0;

当 $z = 0$ 时, $\sigma(z) = 0.5$;

c. 操作:

a. 可以把线性回归的输出 $z = \vec{w}\vec{x} + b$ 作为 Sigmoid 函数的输入, 得到一个介于 0 和 1 之间的概率值

b. 描述: 我们可以先用线性回归确定的方法学习并确定 w 、 b 参数, 然后将得到的 z (也称作**对数几率**) 作为线性回归的输出而当作逻辑回归的输入 (值得注意, 我们在优化 w 和 b 时使用的方法不再均差成本函数, 而是交叉熵成本函数 (损失函数), 该术语我等会再去详细了解)

v. 决策边界

a. 阈值:

我们可以定义阈值为0.5 (当然其他的也可以, 看情况进行决策), 那么 $f_{\vec{w},b}(\vec{x}) \geq 0.5$? yes
—— $y - \hat{y} = 1$; no—— $y - \hat{y} = 0$

b. 决策边界的求取:

对于 $f_{\vec{w},b}(\vec{x}) \geq 0.5$ 即 $g(z) \geq 0.5$ 即线性拟合多项式大于等于0, 此时预测值 $\hat{y}$ (即 $y - \hat{y}$)等于1
有趣的是, 我们在阈值为0.5时, 即使线性回归的函数(多特征也没关系)的值(即输入逻辑函数的 z 值)为零时, 得到的解析式就是我们所需要的决策边界。显然决策边界的可视化曲线依赖于我们进行的线性回归函数

3. 逻辑回归的成本函数:

引出我的初问题: 为什么不能沿用线性回归的MSE损失函数(均方误差成本函数)?

解答: 当我们把 $y - \hat{y}$ 代入MSE损失函数 $L = (y - (y - \hat{y}))^2$ 后, 损失 L 相对于参数 w 的关系就变得非常复杂。由于Sigmoid(逻辑函数)是一个非线性函数, L 关于 w 的二阶导数(Hessian矩阵)不能保证始终是正定的, 这意味着损失函数的“曲率”会变化, 从而产生多个极小值点(非凸)。在具体操作中, 这会导致一个严重的问题: 我们不妨文绉绉地引用一个术语——**梯度消失 (Vanishing Gradient)**

- 其实以上的数学机理我一点都不熟悉, 其中涉及微积分梯度的知识, 以及Hessian矩阵的知识, 这一点我们以后再聊
- 半年后来看: 其实没有那么复杂, 本质在于MSE求Sigmoid函数损失计算梯度时的偏导数呈对称结构, 这导致如果标签是1, 预测是0.99和预测是0.01的梯度的数量级都很小, 甚至于说你0.01的梯度算出来可能都小于0.6的梯度, 这显然不合理

i. 交叉熵损失函数 (对数损失):

先定义单个损失函数元有:

如果 $y^{(i)} = 1$, 则:

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -\log(f_{\vec{w},b}(\vec{x}^{(i)}))$$

如果 $y^{(i)} = 0$, 则:

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$$

显然这样的拟合就比较的美丽

当然我们可以合并简化一下使我们的拟合更加美丽

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = (1 - y^{(i)})[-\log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))] + y^{(i)}[-\log(f_{\vec{w},b}(\vec{x}^{(i)}))]$$

所以简单来说我们需要的交叉熵函数就是每一部分的损失函数的和

ii. 梯度下降的实现: 类比于线性回归中的参数, 具体操作我会在实践操作部分详细展示

4. 概率论与信息论浅谈

i. 最大似然估计 (MLE):

a. 简单来说:最大似然估计就是找那个让“已发生事实”出现概率最大的参数值, 例如:

问题:有一个硬币, 扔了10次, 结果出现了 7次正面, 3次反面

这个硬币是公平的吗 (正反面概率各50%) ? 还是作弊的硬币?

两种假设:

假设A: 硬币是公平的, 正面概率 $p=0.5$

假设B: 硬币作弊, 正面概率 $p=0.7$

显然:假设B在这种情况下可能更大

b. 简单的数学理解: 存在数据D,和参数theta, 似然函数

$$L(\theta) = P(D|\theta)$$

表示在theta参数下观察到D数据的概率, 我们寻找一个theta让这个L函数最大, 即寻求最大似然

c. 与机器学习的联系:

在分类问题上: 就是不断更新参数使预测概率最大

比如我们最小化交叉熵损失函数的过程其实就是寻求最大似然的过程, 下面在熵的笔记后有详细的介绍

ii. 熵:

a. 熵是用来衡量事件不确定性的量:

$$H(P) = - \sum_{i=1}^n p_i \log_2(p_i)$$

iii. 最终联系:

根据事件数据写出似然函数 $L(\theta)$, 再将似然函数取负对数即

$$-\log L(\theta)$$

这个式子可以展开化成

$$- \sum (\text{频数}) \log(\text{预测频率})$$

又

$$\frac{\text{频数}}{N} \approx p_i$$

则

$$-\log L(\theta) = -N \sum_{i=1}^n p_i \log_2(p_i)$$

即

$$-\frac{1}{N} \log L(\theta) = -\sum_{i=1}^n p_i \log_2(p_i)$$

常数N不改变优化方向，所以最大似然就是最小化右边的交叉熵函数，在机器学习里面就是最小化我们的 *CrossEntropy*

决策树

1. 定义:

- i. 组成部分：根节点（第一个问题）；内部节点（中间问题）；叶节点（最终决定）
- ii. 构建过程：原始数据分析——选择最佳问题——分割数据——重复——原始数据分析...
直到所有样本属于同一类，无法继续分割，达到预设深度停止

2. 选择最佳问题:

- i. 关键指标：Purity（纯度），尽可能让每个子节点同一类样本多
- ii. 常用指标：
 - a. 信息增益：

信息增益 = 父节点的不纯度 - 子节点的不纯度加权平均

当信息增益越大越好

- b. 基尼不纯度：衡量随机选两个样本，它们类别不同的概率，值越小越好（0表示完全纯净）
- c. 信息增益率

3. ID3 决策树学习算法:

- i. 从根节点开始，所有数据都在根节点
- ii. 计算当前节点中所有特征的信息增益
- iii. 选择信息增益最大的特征作为分割特征
- iv. 用该特征的每个取值创建分支
- v. 对每个分支，重复步骤2-4，直到：
 - a) 所有样本属于同一类别（纯节点）
 - b) 没有更多特征可用
 - c) 达到预定义停止条件

4. 实例详见pdf文档

过拟合问题

1. 定义（具有高方差）：

过拟合是指机器学习模型在训练数据上表现过于优秀，以至于学习了训练数据中的噪声（Noise）和细节（Detail），而不是数据背后的普遍规律（General Pattern）。这导致模型在训练集上误差非常小，但在新的、未见过的数据（测试集或真实世界数据）上表现很差，即泛化能力（Generalization Ability）很弱

2. 欠拟合（具有高偏差）：

欠拟合是指机器学习模型过于简单，无法捕捉训练数据中的基本规律（Underlying Pattern）或趋势（Trend）。这导致模型不仅在训练数据上表现不佳，在新的、未见过的数据上表现同样很差。它的泛化能力也很弱，但原因是“没学会”而不是“学错了”

3. 过拟合的解决方法：

- i. 获取更多数据：这是最有效的方法之一。更多的数据能让模型看到更全面的情况，不容易被个别噪音带偏
- ii. 数据增强（Data Augmentation）：如果无法获取新数据，可以对现有数据进行一些变换来创造新数据（如图像的旋转、裁剪、加噪声；文本的同义词替换等）
- iii. 简化模型：
 - 选择参数更少、更简单的模型
 - 对于决策树，进行剪枝（Pruning），减少树的深度或叶子节点数
 - 对于神经网络，减少网络层数或神经元数量
- iv. 正则化：
 - 在损失函数中增加一个惩罚项，用来限制模型参数的大小（如 L1 正则化、L2 正则化），迫使模型变得简单
 - L1 (Lasso)：倾向于产生稀疏权重，可用于特征选择
 - L2 (Ridge)：使权重参数普遍接近于零但不为零
- v. Dropout（主要用于神经网络）：
 - 在训练过程中，随机地“丢弃”网络中的一部分神经元，迫使网络不依赖于任何单个神经元，从而学习到更鲁棒的特征
- vi. 早停（Early Stopping）：
 - 在训练神经网络时，监控验证集上的表现。一旦验证集上的误差开始上升，就立即停止训练，防止模型过度优化训练集
- vii. 交叉验证（Cross-Validation）：
 - 帮助更准确地评估模型的泛化能力，并辅助进行超参数调优，找到最不容易过拟合的模型参数
- viii. 集成学习（Ensemble Methods）：
 - 如 Bagging（例如随机森林），通过训练多个模型并综合它们的预测结果，可以有效降低过拟合风险

4. 正则化

i. 基本理解：

在优化目标（损失函数）中增加一个惩罚项，用于限制模型参数的大小和复杂度，通过这种方式，我们迫使模型变得更简单、更平滑，从而提升其泛化能力

正则化后的损失函数：在误差基础上，加上一个对参数本身的惩罚。

ii. 操作：

对于J函数,这里我们取用更权威的写法：

$$J_{regularized}(w) = Loss(x, y, w) + \lambda R(w)$$

$Loss(x, y, w)$: 原来的损失（如均方误差、交叉熵损失）

λ (lambda): 正则化强度超参数。这是我们需要调节的

$\lambda = 0$: 正则项失效，模型可能过拟合

$\lambda \rightarrow \infty$: 惩罚过大，所有参数被压向0，模型会变得极其简单，导致欠拟合

$R(w)$: 正则化项，是参数 w 的函数。不同的 $R(w)$ 定义了不同的正则化方式

通俗解释：一般地，高次项的参数w一般较大，我们在惩罚项将 λ 调得很大，则在最小化J函数的过程中，相当于我们在告诉模型，你也要过拟合，你想把高次项参数调得很大以此来拟合每一个训练数据，那么它就要相应的付出代价——即惩罚项会反向增大J函数，告诉模型我们已经设置了牵制

注意:在这里我偷懒没有写下正则化在线性回归和逻辑回归中的具体表达形式，不过我会在实践操作中再次加深理解

初识深度学习

1. 引言：需求预测

- 数据准备：神经网络的“燃料”

首先，收集了过去几周的历史数据，把它整理成一张表格：

日期	温度	周末	促销	销量
7月1日	28	否	是	210
7月2日	30	是	否	245
7月3日	25	否	否	150
...	...	...	...	...
7月15日	33	是	是	310

- 上表中：温度：是连续数值（28, 30, 25...）；周末：是分类数据，我们用 1 代表“是”，0 代表“否”；是否有促销：同样，用 1 代表“是”，0 代表“否”；实际销量：这是我们想要预测的目标值，也称为标签（Label）

在神经网络中：

温度、是否周末、是否有促销 这三个特征被称为 输入（Input） 或 特征（Feature），预测销量 被称为 输出（Output）

- 构建神经网络模型：设计一个“大脑”

我们现在要设计一个最简单的神经网络来处理这个问题。这个网络的结构如下：

这个网络的工作流程：

- i. 输入层（Input Layer）：有3个神经元，分别接收“温度”、“是否周末”、“是否有促销”这三个数值
 - ii. 隐藏层（Hidden Layer）：我们暂时只设计一层，里面有若干个神经元（比如4个）。每个神经元都会与输入层的每一个神经元相连
 - iii. 输出层（Output Layer）：只有1个神经元，它接收隐藏层传来的信息，最终计算并输出一个连续的数值，即我们预测的销量
 - iv. 关键概念：权重（Weights）和偏置（Bias）：每条连接线都有一个 权重（W）。可以理解为这个输入特征的重要程度。例如，连接“温度”和隐藏层神经元的权重可能很大，因为温度对销量影响很大；而“促销”的权重可能稍小。每个神经元（除了输入层）还有一个 偏置（b）。可以理解为这个神经元的激活难易程度
- 训练神经网络：“学习”的过程：
模型一开始是“无知”的，所有的权重（W）和偏置（b）都是随机初始化的。它第一次看到数据（比如：温度30，周末1，促销0）时，会胡乱计算出一个销量（比如算出10杯），这和真实值（245杯）相差甚远

训练就是不断纠正这个错误的过程：

- i. **前向传播**（Forward Propagation）：数据从输入层传到输出层，计算出预测值
隐藏层每个神经元的计算：

$$\text{输出} = \text{激活函数}((\text{温度} * W1) + (\text{周末} * W2) + (\text{促销} * W3) + b)$$

输出层神经元计算：

$$\text{预测销量} = (\text{隐藏神经元1输出} * W4) + (\text{隐藏神经元2输出} * W5) + \dots + b - output$$

- ii. 计算损失（Loss Calculation）：比较预测销量和真实销量之间的差距。常用均方误差（MSE）作为损失函数（Loss Function）
这个Loss值衡量了模型当前有多“差”
- iii. **反向传播**（Backpropagation）：这是神经网络学习的核心。算法会从输出层开始，反向计算这个Loss是如何由每一层的权重和偏置造成的。它精确地计算出每个权重（W）和偏置（b）对最终误差应负多少“责任”（即梯度）
- iv. **优化更新**（Optimization）：使用优化器（如梯度下降），根据上一步计算出的“责任”大小，微调（更新）所有的权重和偏置

原则是：对误差贡献大的参数，就多调整一点；贡献小的，就少调整一点

$$\text{新权重} = \text{旧权重} - \text{学习率} * \text{梯度}$$

v. **迭代** (Epoch):

将历史数据中的所有样本（例如过去30天的数据）全部“喂”给网络完成一次“前向传播->计算损失->反向传播->更新权重”的过程，称为一个 Epoch。训练通常需要很多个Epoch，直到模型的预测误差收敛到一个很小的值，不再明显下降为止

2. **计算机视觉方面浅谈**

设置多层神经网络，不断扩大感知视野，从定向的边缘线段，再到局部特征再到最终的辨识对象，一步一步在隐藏层中完成运算，最后输出结果

3. 基本概念:

- **去激活值**: 可以理解为是该层神经网络的输入
- **激活函数**: 将去激活值运算成为这一层激活值的函数
- **激活值**: 这一层的输出

思考：目前我见到的激活函数还仍然只有sigmoid函数；当然我们会在以后的操作中遇到其他的激活函数，但无一例外这些激活函数都是对数据进行了一个非线性处理，而很少遇到线性处理：

简答：我们假设这一百层的神经网络全部采取线性处理，那么我们会发现这最后的任然是一个线性的函数，那么这就失去了深度的意义，即不论一百层还是一万层，我们的多层感知机不如改成一个拥有超复杂系数的单层感知机

4. **常用几大层**:

层类型	主要用途	特点	常见应用
卷积层	特征提取	局部连接、权重共享	图像处理、计算机视觉
池化层	降维	减少计算量、平移不变性	所有CNN网络
循环层	序列建模	记忆历史信息	NLP、时序分析
注意力层	重要度加权	动态权重分配	Transformer、机器翻译
归一化层	稳定训练	加速收敛、防止梯度问题	深层网络
丢弃层	防止过拟合	随机失活神经元	所有网络类型
嵌入层	向量表示	离散到连续映射	NLP、推荐系统
转置卷积	上采样	生成高分辨率输出	GANs、语义分割
全连接层	特征组合与分类	全局连接、参数量大、灵活性强	网络末端分类器、传统神经网络

5. 重申 **框架** 的重要性:

i. **自动微分** (Automatic Differentiation - Autodiff):

这是框架最核心、最革命性的功能。 它解决了深度学习中最繁琐、最容易出错的数学问题——梯

度计算。只需要定义前向传播（网络结构），框架会自动构建计算图，并通过反向传播算法精确计算所有参数的梯度

ii. 高性能计算抽象：

框架将复杂的底层计算（如GPU并行计算、矩阵运算优化）封装起来。用简单的 `model(data)` 就能触发高度优化的C++/CUDA代码，而无需关心如何将计算分配到成千上万的GPU核心上

iii. 预构建的模块库：

框架提供了丰富的、经过严格测试的工具（如各种网络层、损失函数、优化算法）。我们可以像搭积木一样快速构建和实验不同的模型架构，而不用重复造轮子

iv. 硬件无缝对接：

框架处理了硬件差异。同一段PyTorch代码，只需简单指定 `device='cuda'`，就能在CPU或GPU上运行，无需修改底层逻辑

v. 工具生态系统：

框架背后是一整套工具链，用于数据加载（DataLoader）、可视化（TensorBoard）、模型部署（TorchScript, TensorFlow Serving）等。这些工具极大地提升了从研发到部署的全流程效率

vi. 极大弱化了基础数学功底，提高效率，增加容错

这里我找了一段从零开始实现nn的代码块；具体内容和语法很多我都看太不懂，写到这里纯粹是对比一下框架极大简化代码的作用，如果有时间的话，我再对从零开始构建的语句进行研究，但显然太复杂了。框架二十行可以解决的问题，零开始用了一百多行

```

import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

# 1. 激活函数及其导数（必须手动实现）
def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True)) # 稳定性处理
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

# 2. 损失函数及其导数（必须手动实现）
def cross_entropy_loss(y_pred, y_true):
    m = y_true.shape[0]
    # 将y_true从标签[0,1,2,...]转换为one-hot编码
    y_true_one_hot = np.eye(10)[y_true]
    log_likelihood = -np.log(y_pred[range(m), y_true] + 1e-8) # 加上小量防止log(0)
    loss = np.sum(log_likelihood) / m
    return loss

# 3. 手动初始化网络参数（权重和偏置）
def initialize_parameters(layer_dims):
    parameters = {}
    for l in range(1, len(layer_dims)):
        parameters['W' + str(l)] = np.random.randn(layer_dims[l], layer_dims[l-1]) * 0.01
        parameters['b' + str(l)] = np.zeros((layer_dims[l], 1))
    return parameters

# 4. 手动实现前向传播
def forward_propagation(X, parameters):
    cache = {'A0': X}
    A_prev = X.T # 注意维度处理，这里是个大坑
    L = len(parameters) // 2 # 层数

    for l in range(1, L):
        Z = parameters['W' + str(l)].dot(A_prev) + parameters['b' + str(l)]
        A = relu(Z)
        cache['Z' + str(l)] = Z
        cache['A' + str(l)] = A
        A_prev = A

```

```

# 输出层不用ReLU，用Softmax
ZL = parameters['W' + str(L)].dot(A_prev) + parameters['b' + str(L)]
AL = softmax(ZL.T).T # 调整维度
cache['Z' + str(L)] = ZL
cache['A' + str(L)] = AL
return AL, cache

```

5. 手动实现反向传播（这是最复杂、最容易出错的部分！）

```

def backward_propagation(AL, Y, cache, parameters):
    grads = {}
    m = Y.shape[0]
    L = len(parameters) // 2
    Y = Y.reshape(AL.shape) # 调整维度

    # 输出层的梯度 (dZ^[L])
    dZ = AL - Y # 这是Softmax+交叉熵损失的一个优美性质，推导很复杂

    for l in reversed(range(1, L+1)):
        if l == L:
            # 输出层
            A_prev = cache['A' + str(l-1)]
        else:
            A_prev = cache['A' + str(l-1)]

        grads['dW' + str(l)] = (1/m) * dZ.dot(A_prev.T)
        grads['db' + str(l)] = (1/m) * np.sum(dZ, axis=1, keepdims=True)

        if l > 1:
            # 计算前一层的dZ
            dA_prev = parameters['W' + str(l)].T.dot(dZ)
            dZ = dA_prev * relu_derivative(cache['Z' + str(l-1)]) # 链式法则

    return grads

```

6. 手动更新参数

```

def update_parameters(parameters, grads, learning_rate):
    L = len(parameters) // 2
    for l in range(1, L+1):
        parameters['W' + str(l)] -= learning_rate * grads['dW' + str(l)]
        parameters['b' + str(l)] -= learning_rate * grads['db' + str(l)]
    return parameters

```

7. 训练循环（现在你需要把所有部分拼起来）

```

layer_dims = [784, 128, 10]

```

```

parameters = initialize_parameters(layer_dims)
learning_rate = 0.01
epochs = 10

for epoch in range(epochs):
    for i in range(0, X_train.shape[0], batch_size):
        X_batch
        = X_train[i:i+batch_size]
        Y_batch = y_train[i:i+batch_size]

        # 前向传播
        AL, cache = forward_propagation(X_batch, parameters)

        # 计算损失
        loss = cross_entropy_loss(AL.T, Y_batch) # 注意维度

        # 反向传播
        grads = backward_propagation(AL, Y_batch, cache, parameters)

        # 更新参数
        parameters = update_parameters(parameters, grads, learning_rate)

    print(f"Epoch {epoch}, Loss: {loss}")

```

再谈零开始实现 单向前向传播

这一部分实践内容占多数，所以不过多展示，详情见jupyter的练习之中

不同的层我们都可以手动定义出来，比如用for循环和基本的矩阵的行列计算来进行全连接层（密集层）的定义，通过不断计算激活值和去激活值作为桥梁链接起来，就定义出来了神经网络的sequential()函数，；根据层的作用我们可以灵活运用于语法对每一个层的运算函数给出手动的定义

激活函数概述

常用的激活函数及其特点

1. Sigmoid 函数

- 输出范围： (0, 1)

- 特点：

- 优点： 将输入值平滑地压缩到 $(0, 1)$ 区间，输出可被直观理解为一种“概率”或“开关状态”。在历史上被广泛使用，尤其适用于二分类问题的输出层
- 缺点：
 - i. 梯度消失： 当输入值非常大或非常小时（函数图像的两端），函数的梯度（导数）会接近于0。在反向传播过程中，梯度会通过网络层连乘，导致靠近输入层的梯度变得极小，权重几乎无法更新，网络难以训练。这是 Sigmoid 函数最大的问题
 - ii. 非零中心： 函数的输出恒大于零。这会导致后续神经元的输入全部为正，在反向传播时权重的梯度更新方向会同为正或同为负，产生“之”字形的优化路径，降低收敛效率
 - iii. 计算成本较高： 涉及指数运算。

- 运用：

- 输出层： 二分类问题的输出层，将输出解释为属于正类的概率
- 隐藏层： 现在已不推荐在隐藏层使用，基本被 Tanh 或 ReLU 等函数取代

2. Tanh 函数（双曲正切函数）

- 输出范围： $(-1, 1)$

- 特点：

- 优点：
 - i. 零中心： 输出以0为中心，其均值是0。这改善了 Sigmoid 非零中心带来的问题，通常能使收敛速度更快
 - ii. 特性与 Sigmoid 类似： 同样是平滑的S型曲线
- 缺点：
 - i. 梯度消失： 虽然比 Sigmoid 稍好（因为梯度在0附近更陡），但仍然存在梯度消失问题

- 运用：

- 隐藏层： 在 ReLU 普及之前，Tanh 通常优于 Sigmoid，是更受欢迎的隐藏层激活函数
- 输出层： 适用于输出需要在负值和正值之间波动的场景，例如生成模型（如LSTM）的中间状态

3. ReLU 函数（修正线性单元）

- 输出范围： $[0, +\infty)$

- 特点：

- 优点：
 - i. 缓解梯度消失： 在正区域 $(x > 0)$ ，梯度恒为1，彻底解决了梯度消失问题，使得深层网络的训练成为可能
 - ii. 计算高效： 只需判断输入是否大于零，计算速度极快
- 缺点：
 - 1. **Dying ReLU（神经元死亡）**： 在负区域 $(x < 0)$ ，梯度恒为0。一旦一个神经元的大多数输入都使其输出为负，该神经元的权重在训练过程中可能永远无法更新，导致该神经元“死亡”且不再对任何数据做出响应
 - ii. 非零中心： 输出非负

- 运用：
 - 隐藏层：目前最常用、默认的隐藏层激活函数。对于大多数情况，尤其是卷积神经网络和深层前馈网络，效果都非常好

4. Leaky ReLU 函数（带泄露的 ReLU）

- 输出范围： $(-\infty, +\infty)$
- 特点：
 - 优点：
 - i. 解决 Dying ReLU 问题：在负区域有一个小的、非零的梯度 (α)，确保了负输入的信息不会完全丢失，神经元不会彻底“死亡”
 - ii. 保留了 ReLU 的所有优点（计算高效、正区梯度大）
 - 缺点：
 - i. 效果不一致：虽然理论上优于 ReLU，但在某些数据集上的表现并不总是稳定地优于 ReLU
 - ii. 需要选择超参数 α ：
虽然通常设为 0.01，但也是一个需要调整的参数
- 运用：
 - 隐藏层：当担心出现“神经元死亡”问题时，可以尝试使用 Leaky ReLU 代替 ReLU

5. Parametric ReLU (PReLU)

- 特点：与 Leaky ReLU 类似，但负区段的斜率参数 α 不是固定的超参数，而是通过反向传播从数据中学习得到
- 运用：在大型数据集（如 ImageNet）上，PReLU 有时能比 ReLU 获得更好的性能，因为它更灵活。但增加了模型的复杂性和训练成本

6. ELU 函数（指数线性单元）

- 特点：
 - 优点：
 - i. 兼具 ReLU 的优点（正区梯度为1）和 Leaky ReLU 的优点（负区有输出）
 - ii. 接近零均值输出：负区的输出使得函数的均值更接近零，这有助于加速训练
 - iii. 在处理负输入时比 ReLU 和 Leaky ReLU 更平滑
 - 缺点：
 - i. 计算成本较高：负区涉及指数运算，比 ReLU 慢
- 运用：
 - 隐藏层：在某些任务上（如图像分类）被证明比 ReLU 有微小的提升，但因其计算成本，不如 ReLU 使用广泛。

7. Softmax 函数

- 输出范围： $(0, 1)$ ，且所有输出的和为 1

- 特点：
 - 优点： 将一个 K 维的任意实数向量“压缩”到另一个 K 维的实向量中，使得每个元素的范围在 (0,1) 之间，并且所有元素的和为 1。这完美地满足了概率分布的性质
 - 缺点： 仅适用于多分类问题的输出层
- 运用：
 - 输出层： 专门用于多分类问题的输出层。它将每个类别的输出解释为属于该类别的概率

多类别

1. 逻辑回归的泛化:**Softmax函数回归算法**:

数学表达：

$$\sigma(z)_i = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

2. 损失函数:交叉熵损失函数

Python基础语法补充

1. 变量和数据类型

```
# 变量赋值 - 就像给东西贴标签
name = "张三"           # 字符串，用引号包围
age = 25                 # 整数
height = 175.5           # 浮点数（小数）
is_student = True        # 布尔值（True 或 False）

# 打印输出
print("姓名:", name)     # 输出：姓名：张三
print("年龄:", age)      # 输出：年龄：25

# 查看数据类型
print(type(name))        # <class 'str'> 字符串类型
print(type(age))         # <class 'int'> 整数类型
print(type(height))      # <class 'float'> 浮点数类型
```

2. 列表和字典

```
# 列表 - 有序的元素集合
fruits = ["苹果", "香蕉", "橙子"] # 字符串列表
numbers = [1, 2, 3, 4, 5]         # 数字列表
```

```
# 访问列表元素（从0开始计数）
print(fruits[0]) # "苹果" - 第一个元素
print(fruits[1]) # "香蕉" - 第二个元素
```

```
# 字典 - 键值对集合
```

```
person = {
    "name": "李四",
    "age": 30,
    "city": "北京"
}
```

```
# 访问字典值
```

```
print(person["name"]) # "李四"
print(person["age"])  # 30
```

3. 函数定义和使用

```
# 定义函数 - def 函数名(参数):
```

```
def greet(name): # name是参数
    """这是一个问候函数""" # 文档字符串, 说明函数用途
    return "你好, " + name # return 返回结果
```

```
# 使用函数
```

```
message = greet("王五") # 调用函数, 传入参数"王五"
print(message) # 输出: 你好, 王五
```

```
# 另一个例子: 计算面积
```

```
def calculate_area(length, width):
    """计算矩形面积"""
    area = length * width
    return area
```

```
# 使用函数
```

```
result = calculate_area(5, 3)
print("面积:", result) # 输出: 面积: 15
```

4. 类和方法

定义类 - 类是对象的蓝图

```
class Student:
    def __init__(self, name, age): # 初始化方法，创建对象时自动调用
        self.name = name # self代表对象本身
        self.age = age

    def introduce(self): # 方法 - 对象的行为
        return f"我叫{self.name}，今年{self.age}岁"
```

使用类创建对象

```
student1
= Student("小明", 20) # 创建Student对象
print(student1.introduce()) # 调用对象的方法
```

5. 缩进

```
if age > 18:
    print("成年人") # 缩进4个空格
    can_drive = True # 缩进4个空格
else:
    print("未成年人") # 缩进4个空格
```

6. 注释

单行注释

```
"""
```

多行注释

用三个引号包围

```
"""
```

```
def function():
    """文档字符串 - 说明函数用途"""
    pass
```

6. 条件判断

```
age = 20

if age < 18:
    print("少年")
elif age < 60: # else if 的缩写
    print("成年")
else:
    print("老年")
```

7. 细谈类和方法

```
# 定义一个“房子”类 (Class)
class House:
    # __init__ 是一个特殊方法，用于初始化新创建的对象
    # self 指的是“这个对象本身”，后面会详细讲
    def __init__(self, style, bedrooms):
        # 属性 (Attributes)
        self.style = style # 户型
        self.bedrooms = bedrooms # 卧室数量
        self.light_on = False # 灯是否亮着

    # 方法 (Method) - 对象的行为
    def turn_on_light(self):
        self.light_on = True
        print(f"{self.style}房子的灯亮了！")

    def describe(self):
        print(f"这是一栋{self.style}户型的房子，有{self.bedrooms}间卧室。")

# 根据“蓝图”(类)来创建实实在在的“对象”
my_house = House("现代简约", 3) # my_house 就是一个对象
your_house = House("欧式别墅", 5) # your_house 是另一个对象

# 使用对象
my_house.describe() # 输出：这是一栋现代简约户型的房子，有3间卧室。
my_house.turn_on_light() # 输出：现代简约房子的灯亮了！

your_house.describe() # 输出：这是一栋欧式别墅户型的房子，有5间卧室。
# your_house的灯还没开呢
```

我们在机器学习中用到过很多的都是方法

8. ...for...in...语句用法详解

```
# 遍历列表
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
# 输出:
# apple
# banana
# cherry
#-----
# 遍历字符串
word = "Python"
for char in word:
    print(char)
# 输出:
# P
# y
# t
# h
# o
# n
#-----
# 遍历元组
colors = ('red', 'green', 'blue')
for color in colors:
    print(color)
# 输出:
# red
# green
# blue
#-----
# 遍历字典
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}

# 遍历键
for key in person:
    print(key)
# 输出:
# name
# age
# city

# 遍历值
for value in person.values():
    print(value)
```

```
# 输出:
# Alice
# 25
# New York

# 遍历键值对
for key, value in person.items():
    print(f"{key}: {value}")
# 输出:
# name: Alice
# age: 25
# city: New York
#-----
# 同时获取索引和值
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(f"索引 {index}: {fruit}")
# 输出:
# 索引 0: apple
# 索引 1: banana
# 索引 2: cherry

# 指定起始索引
for index, fruit in enumerate(fruits, start=1):
    print(f"第 {index} 个水果: {fruit}")
#-----
# 同时获取索引和值
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(f"索引 {index}: {fruit}")
# 输出:
# 索引 0: apple
# 索引 1: banana
# 索引 2: cherry

# 指定起始索引
for index, fruit in enumerate(fruits, start=1):
    print(f"第 {index} 个水果: {fruit}")
#-----
# 同时遍历多个列表
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]
cities = ['NY', 'LA', 'Chicago']

for name, age, city in zip(names, ages, cities):
```

```

    print(f"{name} is {age} years old and lives in {city}")
# 输出:
# Alice is 25 years old and lives in NY
# Bob is 30 years old and lives in LA
# Charlie is 35 years old and lives in Chicago
#-----
# 遍历数字范围
for i in range(5):
    print(i)
# 输出: 0 1 2 3 4

# 指定起始和结束值
for i in range(2, 6):
    print(i)
# 输出: 2 3 4 5

# 指定步长
for i in range(0, 10, 2):
    print(i)
# 输出: 0 2 4 6 8

# 反向遍历
for i in range(5, 0, -1):
    print(i)
# 输出: 5 4 3 2 1
#-----
# 基本的列表推导式
squares = [x**2 for x in range(1, 6)]
print(squares) # 输出: [1, 4, 9, 16, 25]

# 带条件的列表推导式
even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]
print(even_squares) # 输出: [4, 16, 36, 64, 100]

# 嵌套循环的列表推导式
pairs = [(x, y) for x in range(3) for y in range(3)]
print(pairs) # 输出: [(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)]
#-----
# 基本的列表推导式
squares = [x**2 for x in range(1, 6)]
print(squares) # 输出: [1, 4, 9, 16, 25]

# 带条件的列表推导式
even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]
print(even_squares) # 输出: [4, 16, 36, 64, 100]

```

```

# 嵌套循环的列表推导式
pairs = [(x, y) for x in range(3) for y in range(3)]
print(pairs) # 输出: [(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)]
#-----

# 创建字典
squares_dict = {x: x**2 for x in range(1, 6)}
print(squares_dict) # 输出: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# 基于现有字典创建新字典
original = {'a': 1, 'b': 2, 'c': 3}
doubled = {k: v*2 for k, v in original.items()}
print(doubled) # 输出: {'a': 2, 'b': 4, 'c': 6}
#-----

# 创建集合
unique_squares = {x**2 for x in range(-3, 4)}
print(unique_squares) # 输出: {0, 1, 4, 9}
#-----

# 生成器表达式（惰性求值）
squares_gen = (x**2 for x in range(1, 6))
for square in squares_gen:
    print(square)
# 输出: 1 4 9 16 25
#-----

import itertools

# 排列组合
colors = ['red', 'green', 'blue']

# 排列
for perm in itertools.permutations(colors, 2):
    print(perm)
# 输出: ('red', 'green'), ('red', 'blue'), ('green', 'red'), ...

# 组合
for comb in itertools.combinations(colors, 2):
    print(comb)
# 输出: ('red', 'green'), ('red', 'blue'), ('green', 'blue')

# 无限循环
counter = itertools.count(1)
for i, num in zip(range(5), counter):
    print(num)
# 输出: 1 2 3 4 5
#-----

```


遍历二维列表

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

```
for row in matrix:  
    for element in row:  
        print(element, end=' ')  
    print()
```

输出:

1 2 3

4 5 6

7 8 9

#-----

break 示例

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

输出: 0 1 2 3 4

continue 示例

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

输出: 0 1 3 4

#-----

for 循环的 else 子句（循环正常结束时执行）

```
for i in range(3):  
    print(i)  
else:  
    print("循环正常结束")
```

输出: 0 1 2 循环正常结束

如果循环被 break 中断, else 不会执行

```
for i in range(3):  
    if i == 1:  
        break  
    print(i)  
else:  
    print("这行不会执行")
```

输出: 0

```
#-----  
# 反向遍历列表  
fruits = ['apple', 'banana', 'cherry']  
for fruit in reversed(fruits):  
    print(fruit)  
# 输出: cherry banana apple  
#-----  
# 按排序顺序遍历  
numbers = [3, 1, 4, 1, 5, 9, 2]  
for num in sorted(numbers):  
    print(num)  
# 输出: 1 1 2 3 4 5 9  
  
# 按自定义规则排序  
words = ['apple', 'banana', 'cherry', 'date']  
for word in sorted(words, key=len):  
    print(word)  
# 输出: date apple cherry banana  
#-----
```